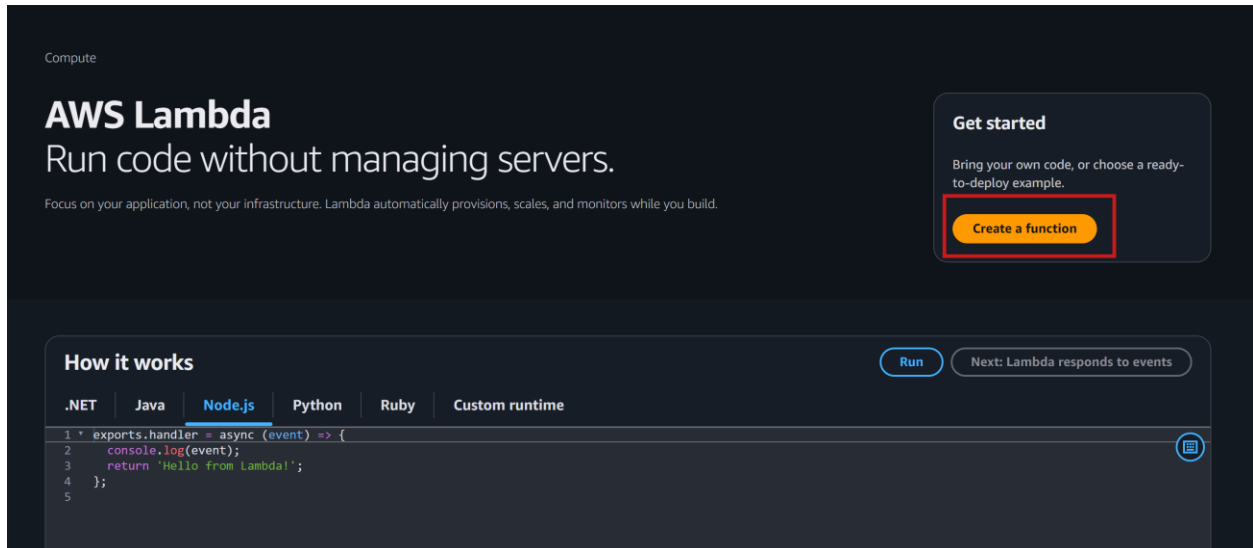


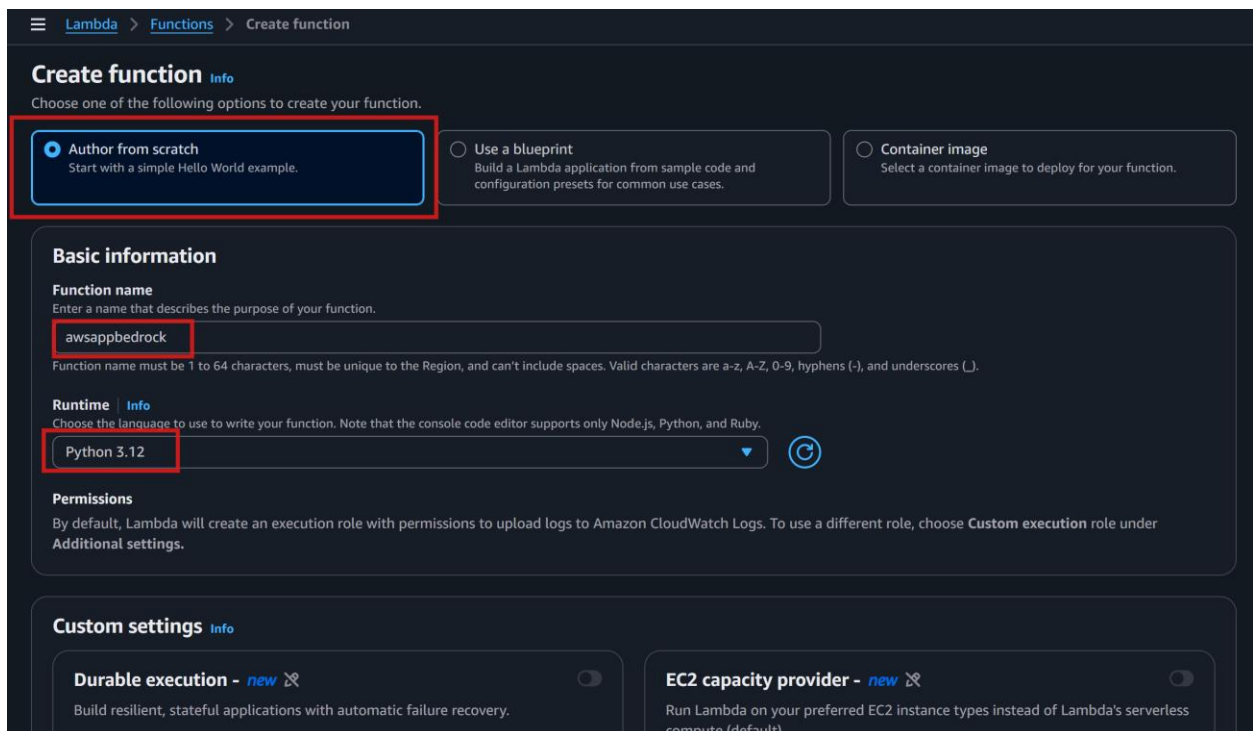
Serverless Blog Generator with AWS Bedrock

AWS services used: Bedrock, Lambda, API Gateway, S3, CloudWatch

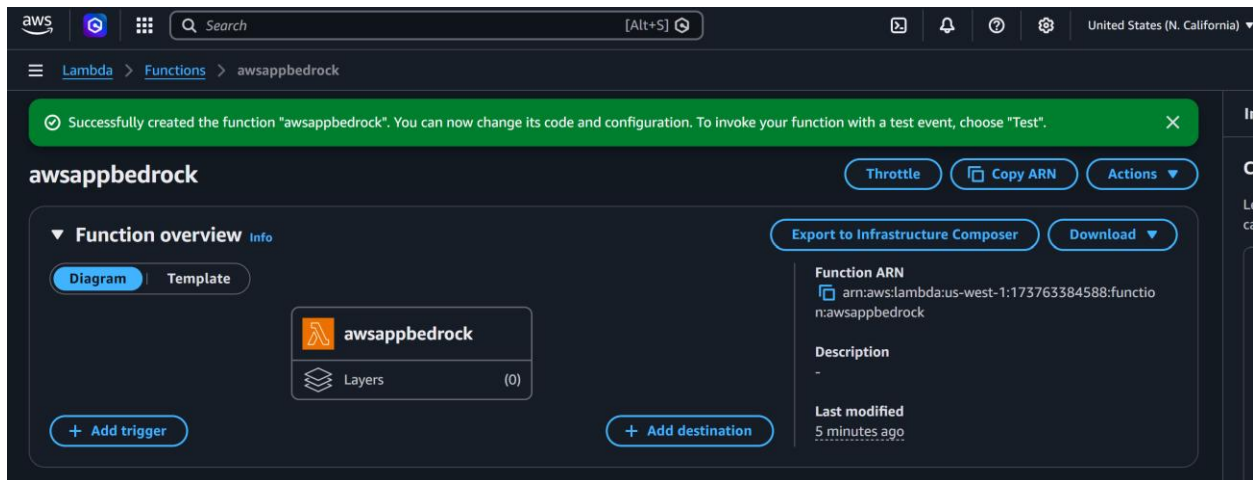
1. Open the AWS Lambda console and click “Create function.”



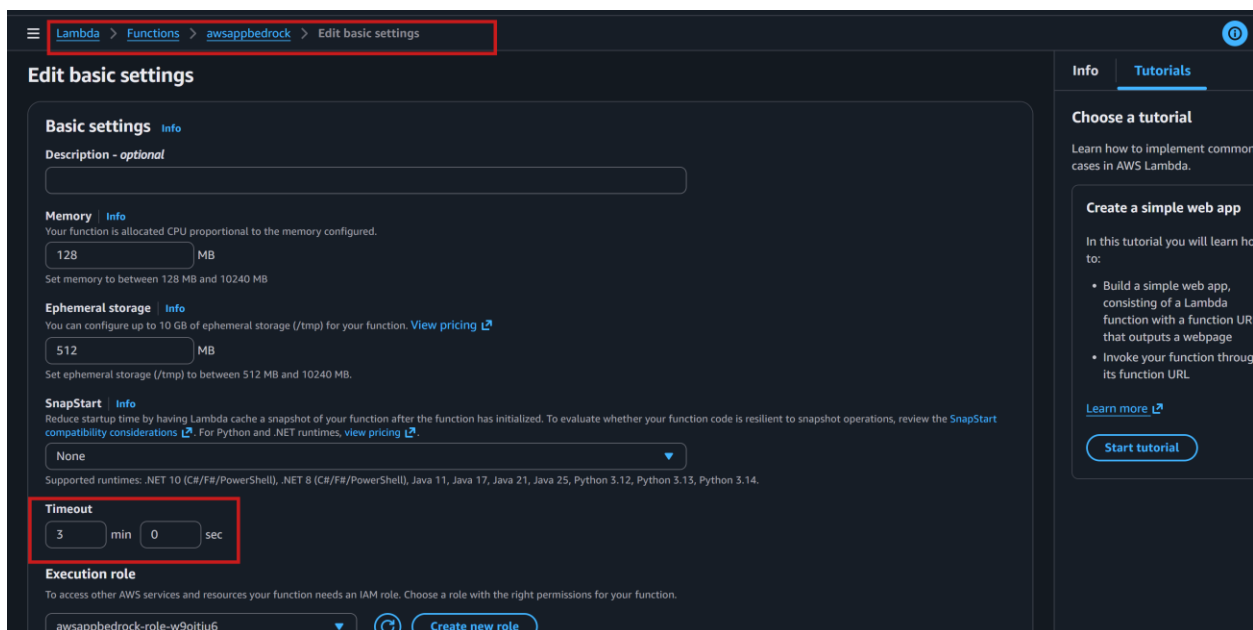
2. Choose “Author from scratch,” name the function, and select the Python runtime.



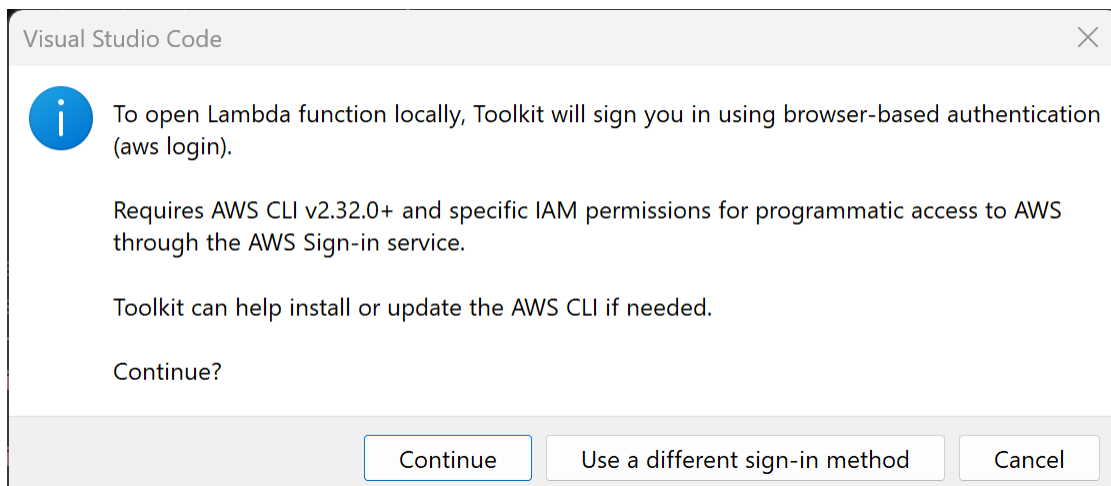
3. Function created successfully.



4. Edit the basic settings to increase the timeout (e.g., 3 min).



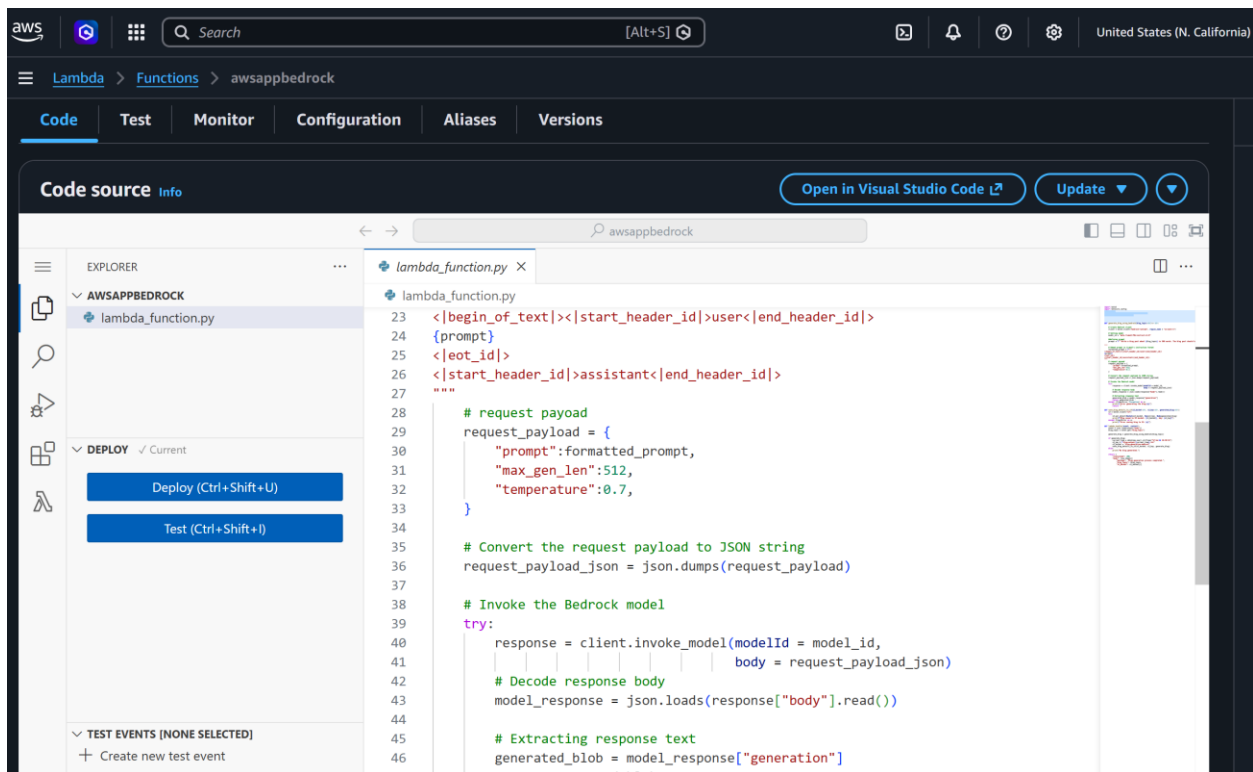
5. Install the AWS Toolkit extension in VS Code and sign in to AWS.



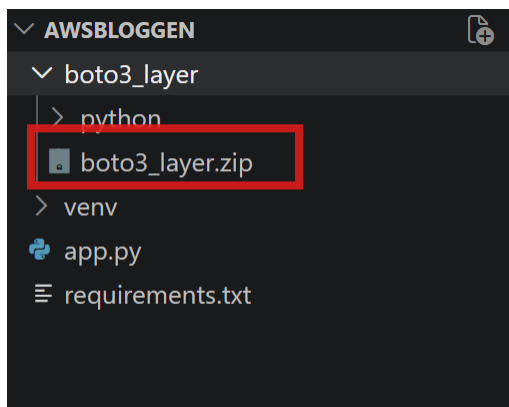
6. Write the Lambda function code that calls Bedrock to generate the blog post.

```
1 import boto3
2 import botocore.config
3 import json
4 from botocore.exceptions import ClientError
5 from datetime import datetime
6
7
8
9 def generate_blog_using_bedrock(blog_topic:str)-> str:
10
11     # Create Bedrock client
12     client = boto3.client("bedrock-runtime", region_name = "us-west-1")
13
14     # Setting model
15     model_id = "meta.llama3-70b-instruct-v1:0"
16
17     #defining prompt
18     prompt = f"""Write a blog post about {blog_topic} in 500 words. The blog post should be engaging, informative, and well-structured. It sh
19
20
21     # Embed prompt in Llama3's instruction format
22     formatted_prompt = """
23     <|begin_of_text|><|start_header_id|>user<|end_header_id|>
24     {prompt}
25     <|eot_id|>
26     <|start_header_id|>assistant<|end_header_id|>
27     """
28
29     # request payload
30     request_payload = {
31         "prompt":formatted_prompt,
32         "max_gen_len":512,
33         "temperature":0.7,
34     }
35
36     # Convert the request payload to JSON string
37     request_payload_json = json.dumps(request_payload)
38
39     # Invoke the Bedrock model
40     try:
41         response = client.invoke_model(modelId = model_id,
42                                     body = request_payload_json)
```

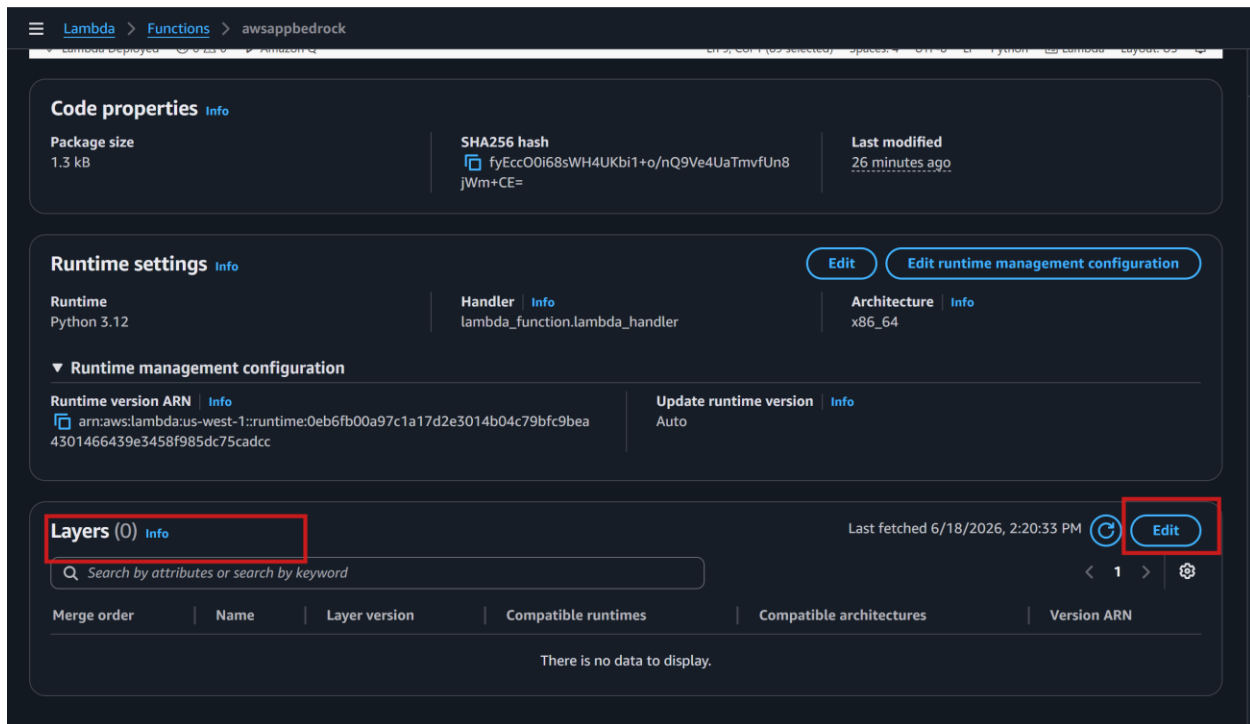
7. Update the code, click “Deploy,” and refresh to confirm it’s live.



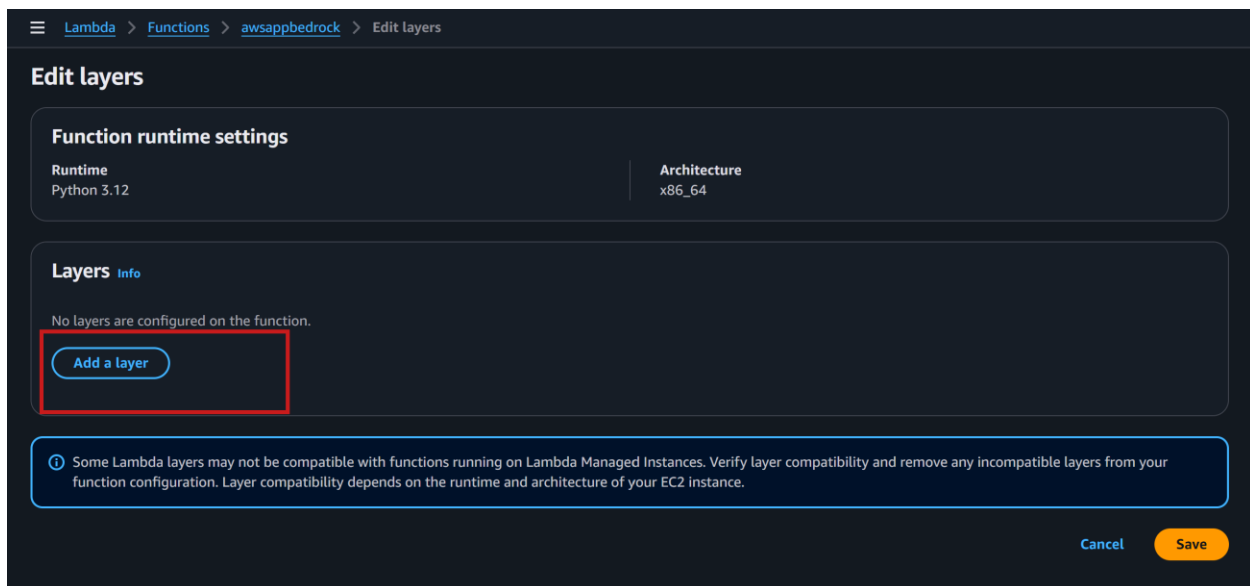
8. Locally, run “pip install boto3 -t python/” and zip the folder as the layer package.



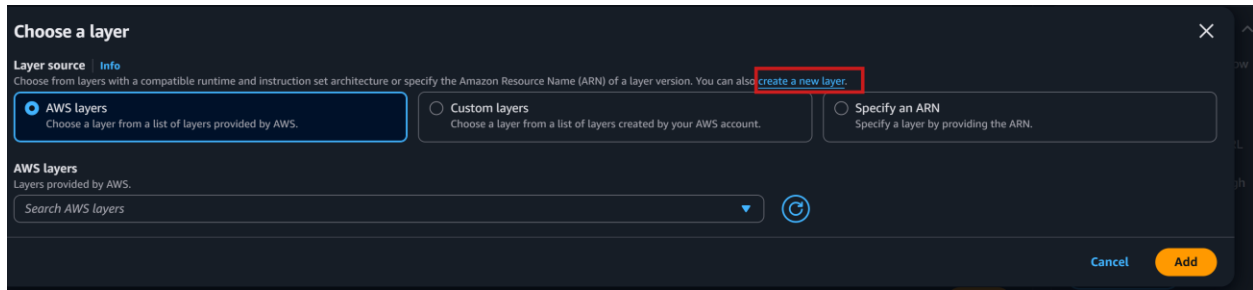
9. The function has no layers yet — click “Edit” to add one.



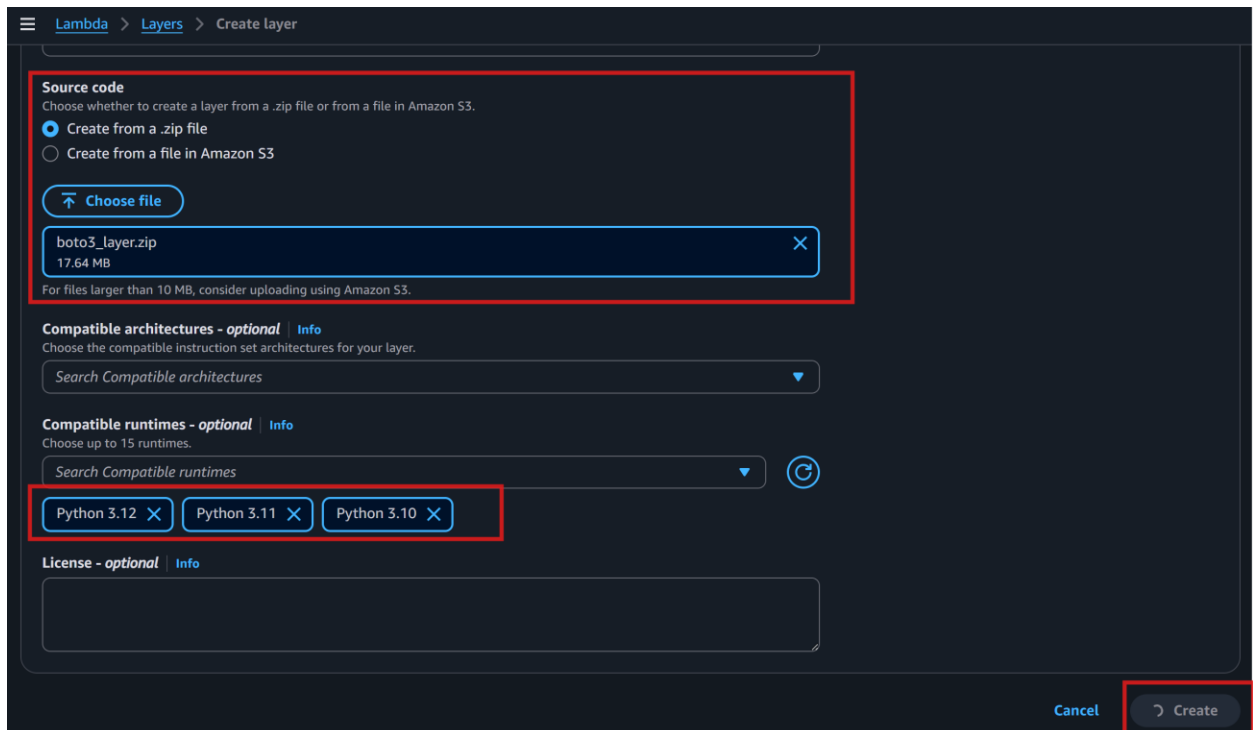
10. In “Edit layers,” click “Add a layer.”



11. Choose a layer source, or click “create a new layer.”



12. Upload the zip file and select the compatible Python runtimes.



13. Layer "boto3_layer" created successfully.

☑ Successfully created layer boto3_layer version 1.

boto3_layer

Delete Download Create version

Version details

Version 1	Version ARN  arn:aws:lambda:us-west-1:173763384588:layer:boto3_layer:1	Description -
Created 31 seconds ago	License -	Compatible runtimes python3.12, python3.11, python3.10
Compatible architectures -		

Versions

Functions using this version

All versions (1)

Last fetched 6/18/2026, 2:39:14 PM 

Search by attributes or search by keyword

Version	Version ARN	Description
1	arn:aws:lambda:us-west-1:173763384588:layer:boto3_layer:1	-

14. Select “Custom layers,” pick the layer and version, then click “Add.”

Choose a layer

Layer source **Info**

Choose from layers with a compatible runtime and instruction set architecture or specify the Amazon Resource Name (ARN) of a layer version. You can also create a new layer.

☐ AWS layers
Choose a layer from a list of layers provided by AWS.

☒ Custom layers
Choose a layer from a list of layers created by your AWS account.

☐ Specify an ARN
Specify a layer by providing the ARN.

Custom layers

Layers created by your AWS account.

boto3_layer 

Version
1 

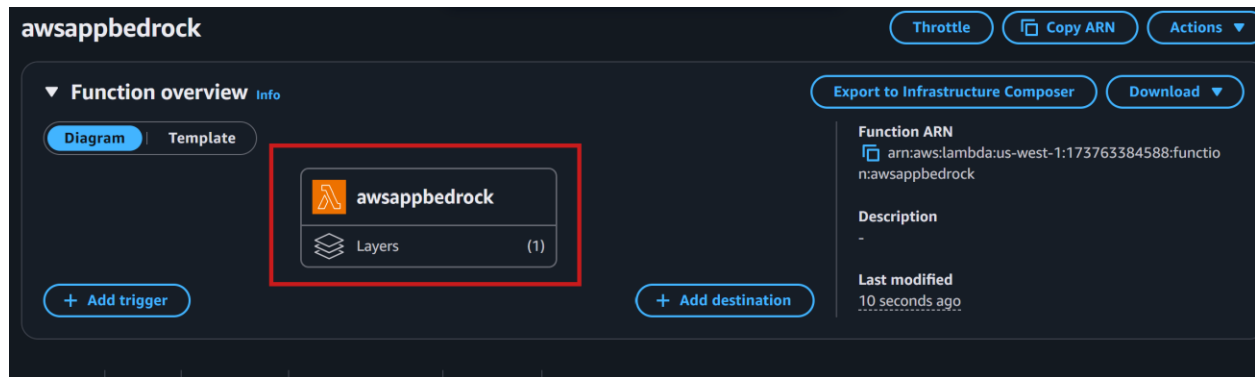
Description
-

Compatible runtimes
Python 3.12, Python 3.11, Python 3.10

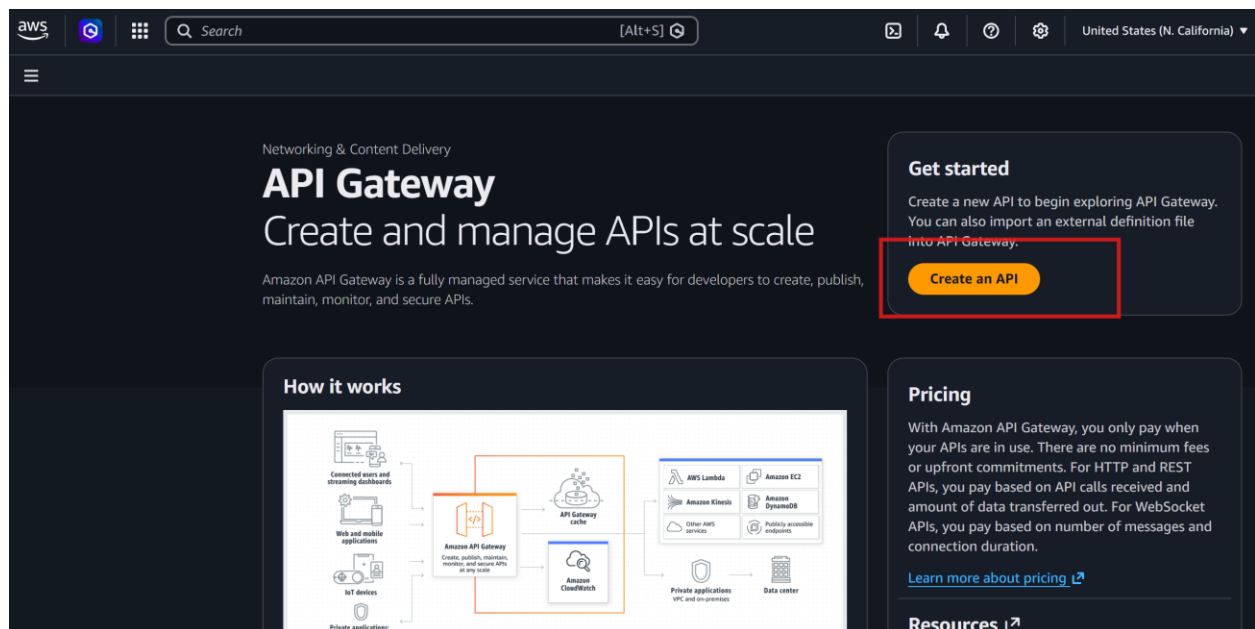
Compatible architectures
-

Cancel Add

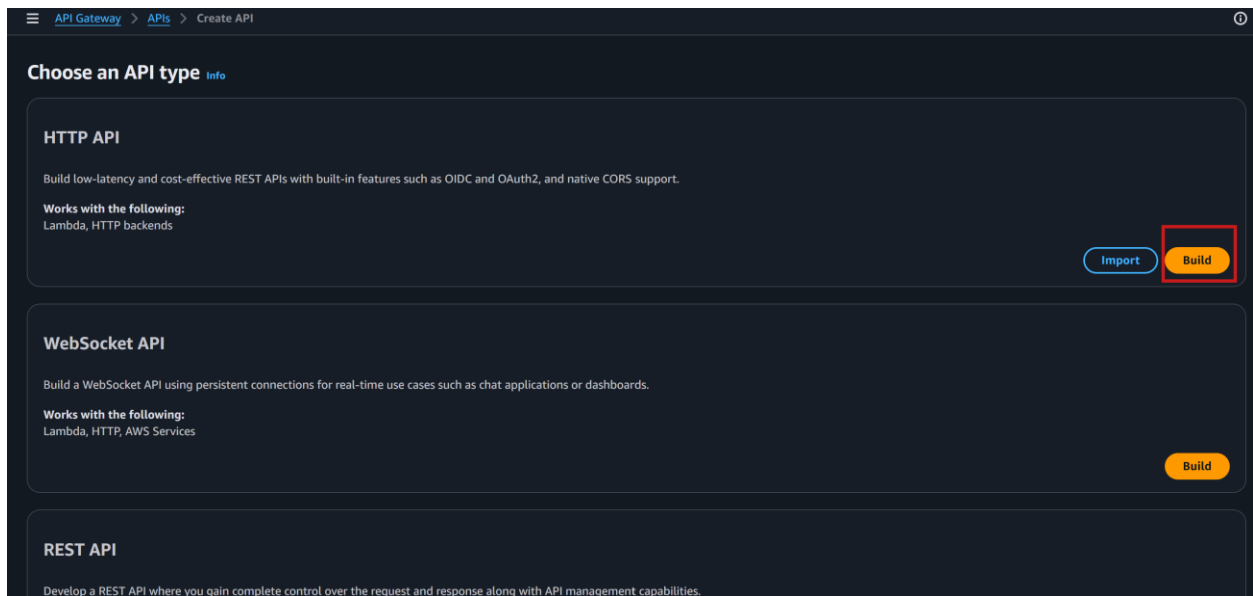
15. The layer is now attached to the function.



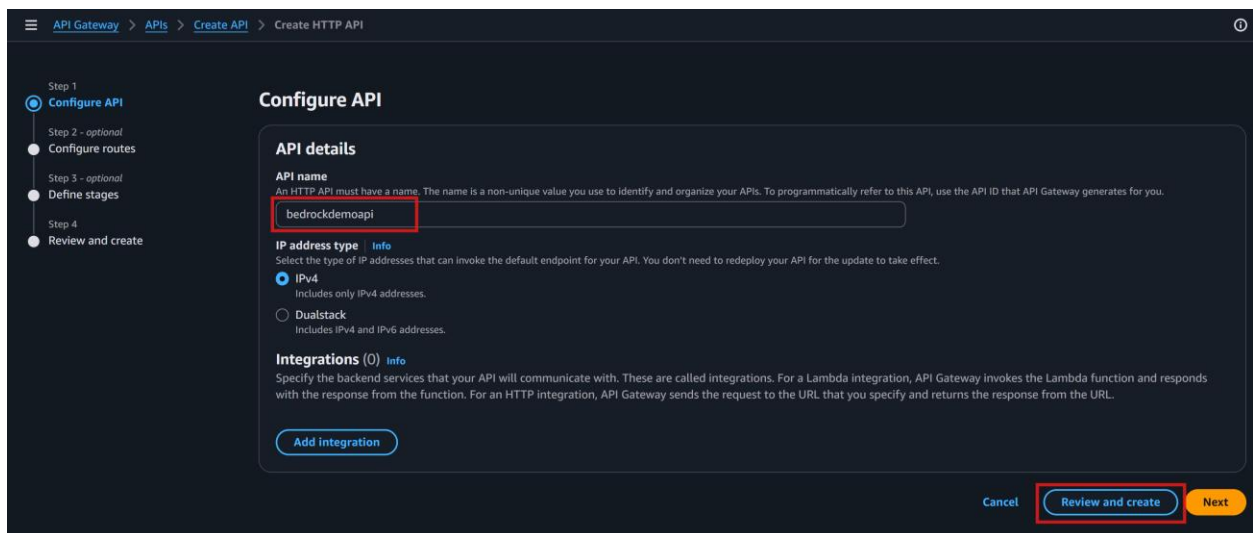
16. Open API Gateway and click “Create an API.”



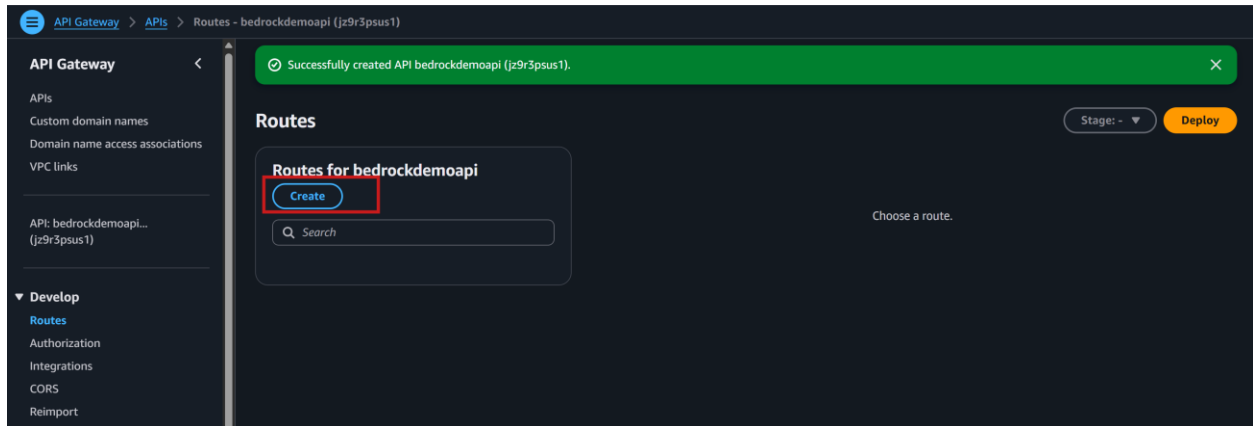
17. Choose “HTTP API” and click “Build.”



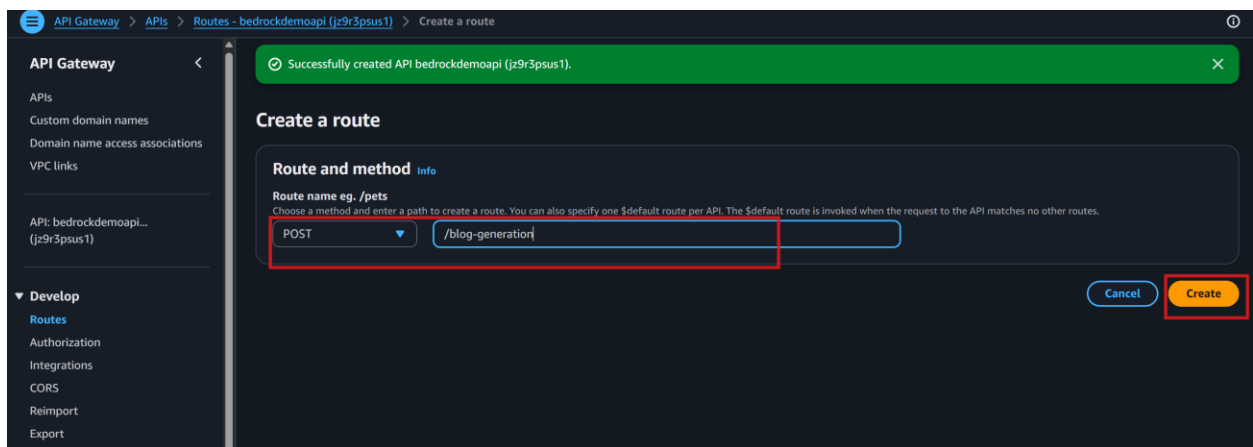
18. Name the API (e.g., bedrockdemoapi) and configure it.



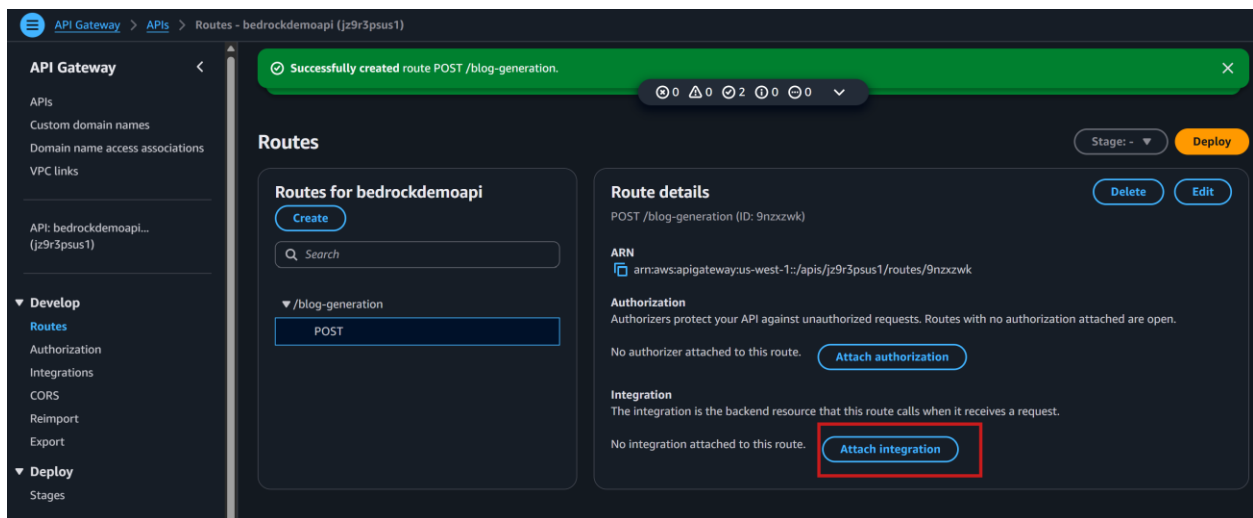
19. API created — click “Create” under Routes to add a route.



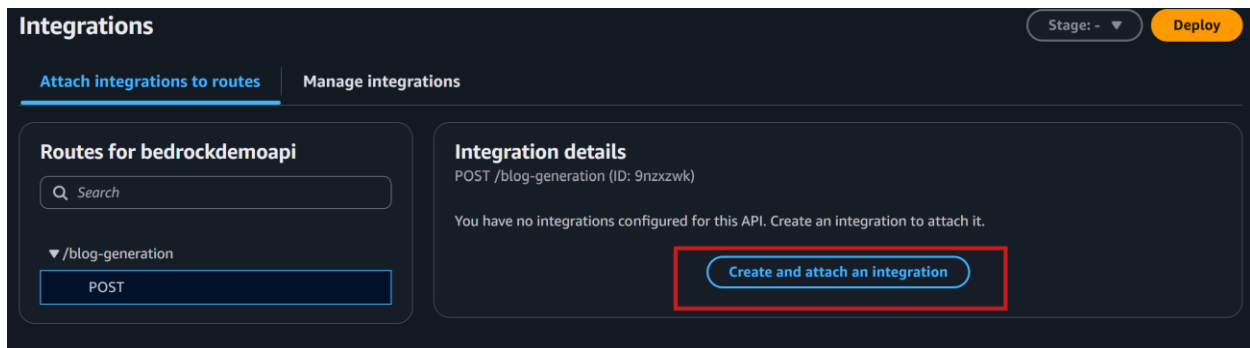
20. Define the route: POST /blog-generation.



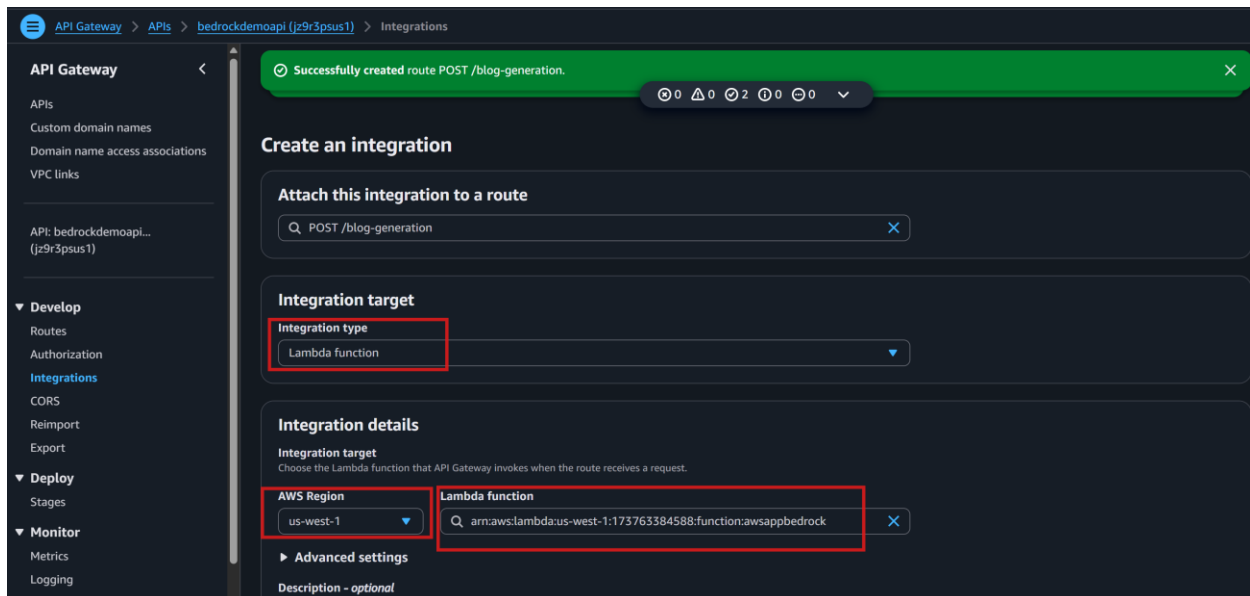
21. Route created — click “Attach integration.”



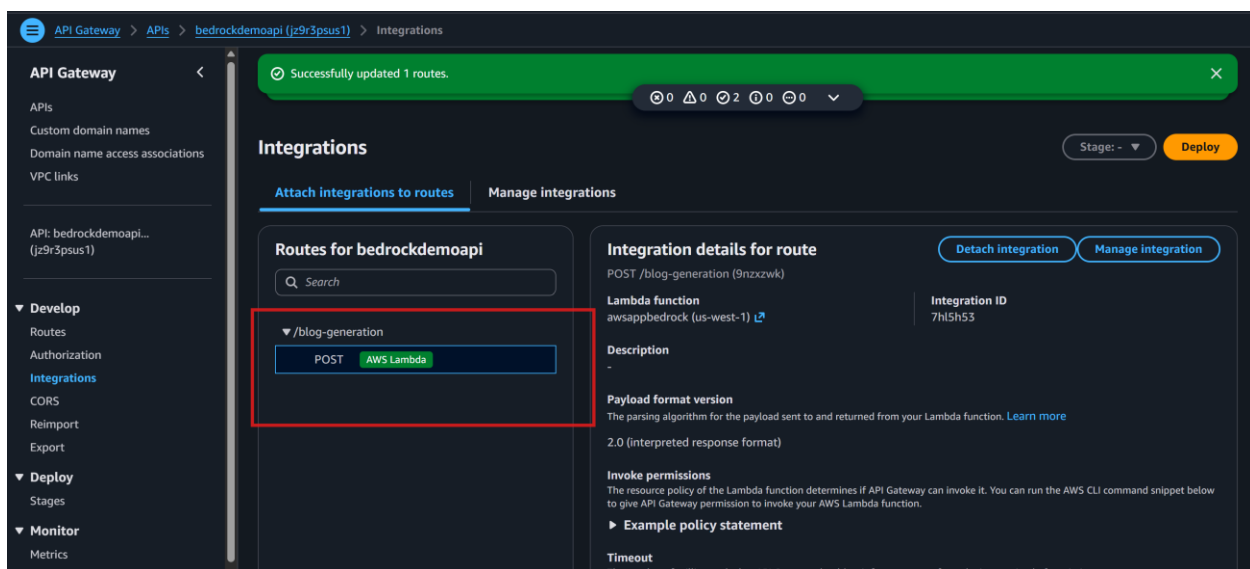
22. Click “Create and attach an integration.”



23. Set integration type to “Lambda function” and select the function created earlier.



24. The Lambda function is now attached to the route.



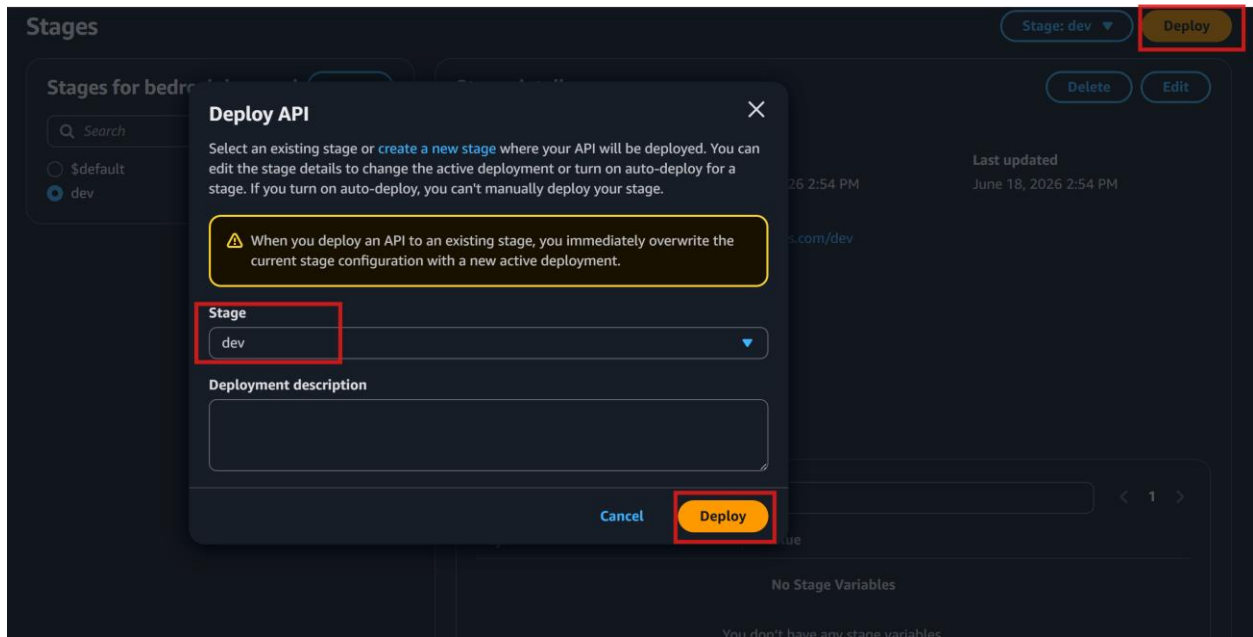
25. Create a deployment stage (e.g., “dev”).

The screenshot shows the 'Create Stage' page in the AWS API Gateway console. The left sidebar contains navigation links for API Gateway, APIs, and Stages. The main content area is titled 'Create Stage' and includes three sections: 'Stage details', 'Stage deployment', and 'Stage variables'. In the 'Stage details' section, the 'Name' field is set to 'dev'. In the 'Stage deployment' section, the 'Enable automatic deployment' checkbox is unchecked, and the 'Deployment' dropdown is set to 'Select deployment'. In the 'Stage variables' section, there is an 'Add stage variable' button.

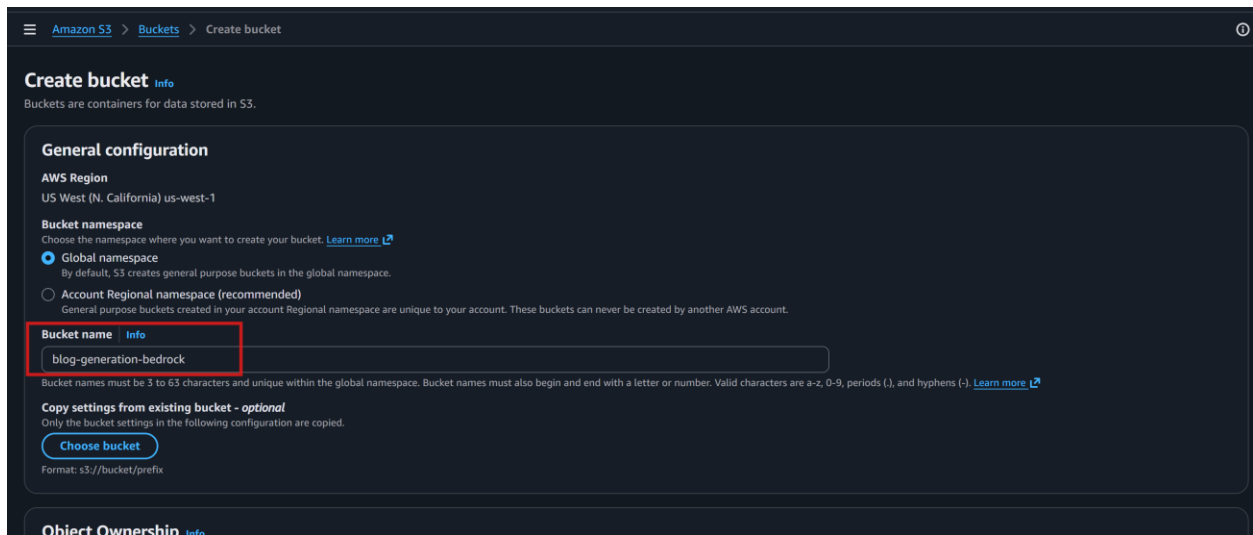
26. Stage created — copy the Invoke URL for the API.

The screenshot shows the 'Stages' page in the AWS API Gateway console. The left sidebar contains navigation links for API Gateway, APIs, and Stages. The main content area is titled 'Stages' and includes a 'Create' button, a search bar, and a list of stages. The 'dev' stage is selected, and its details are shown on the right. The 'Invoke URL' is highlighted with a red box and is the URL <https://jz9r3psus1.execute-api.us-west-1.amazonaws.com/dev>. The 'Attached deployment' section shows 'Automatic Deployment' is disabled.

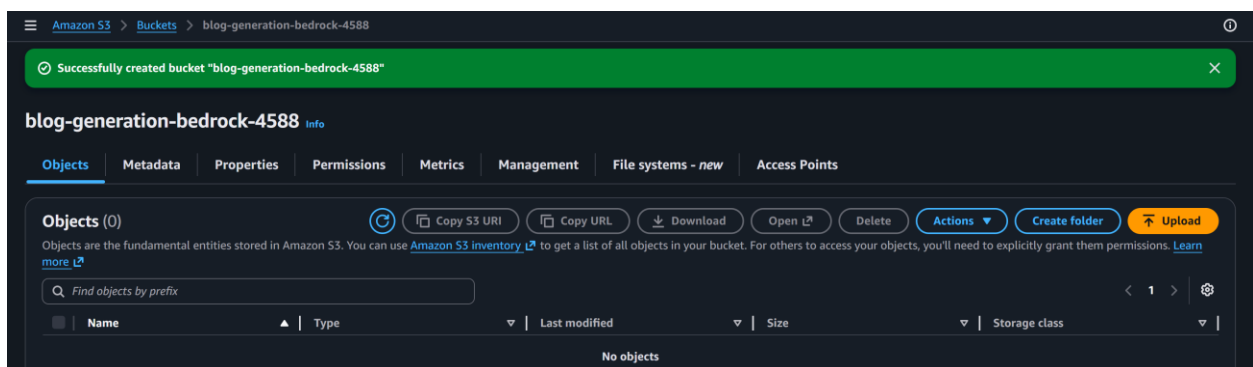
27. Click “Deploy” to push the API to the stage.



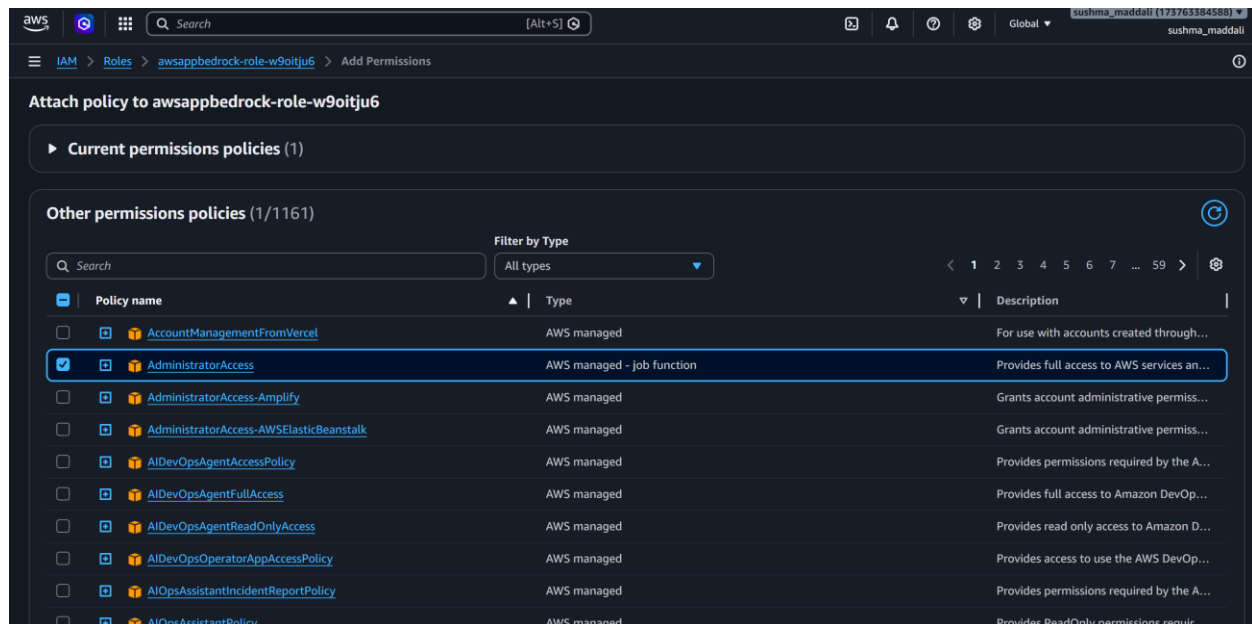
28. In S3, create a bucket to store the generated blogs.



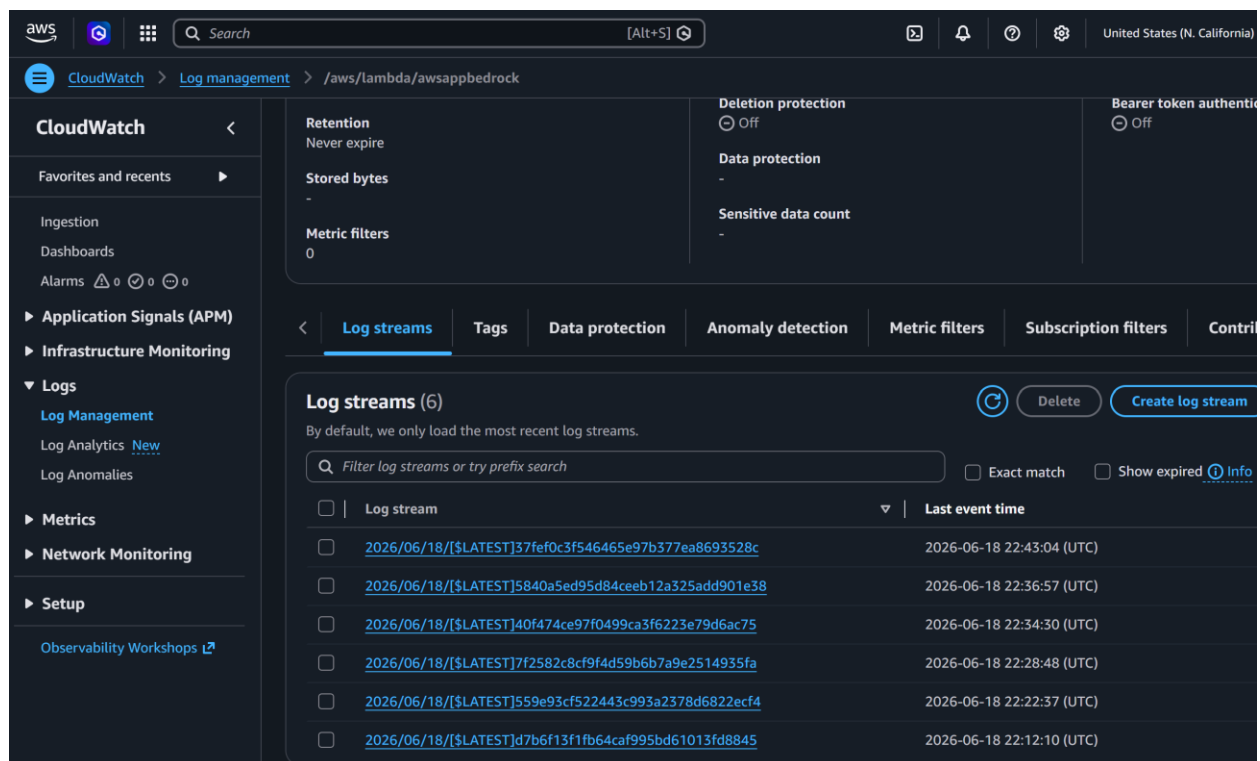
29. Bucket created successfully.



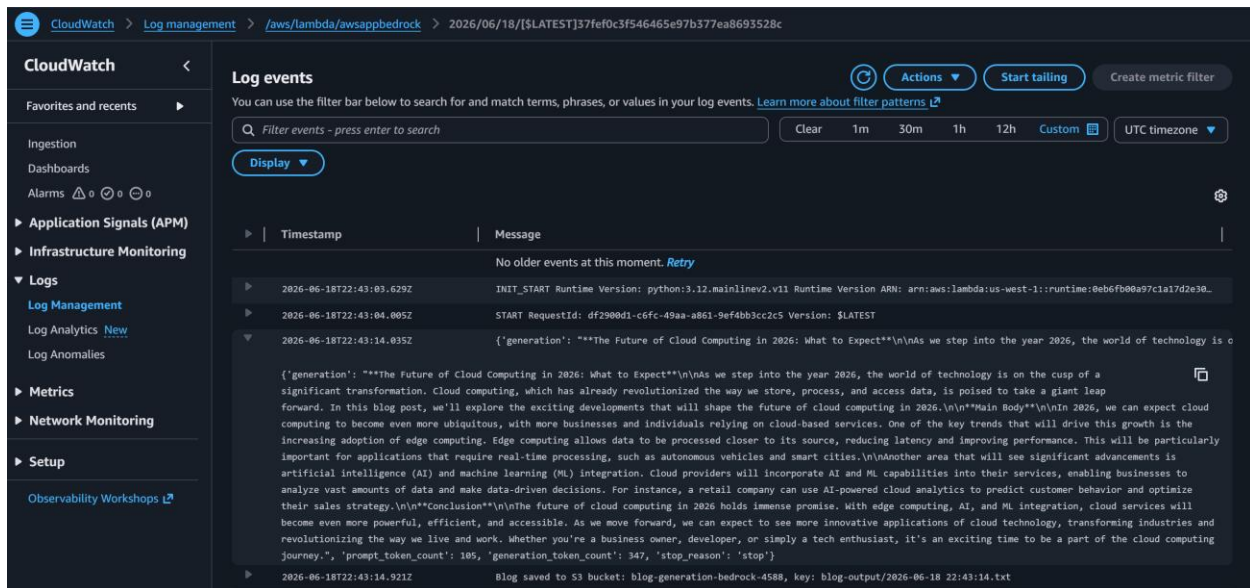
30. Attach the AdministratorAccess policy to the Lambda role so it can call Bedrock.



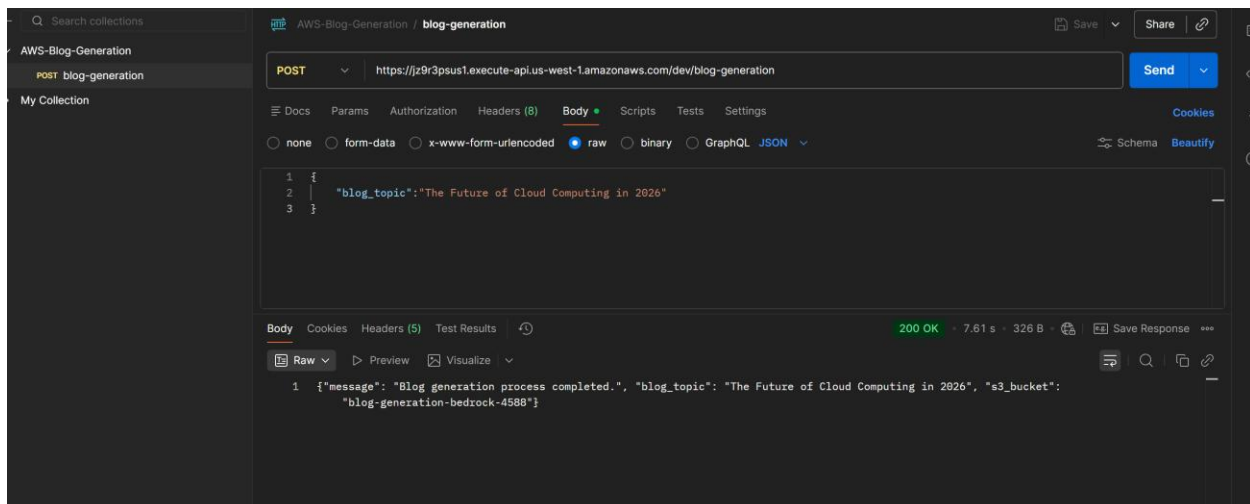
31. View the CloudWatch log streams for the Lambda function.



32. Logs show the generated blog content and confirmation it was saved to S3.



33. Test the API in Postman — POST a blog topic and receive a success response.



34. Generated blog files now appear in the S3 bucket.

Amazon S3

> Buckets

> blog-generation-bedrock-4588

> blog-output/

Amazon S3

<

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

▼ Files

File systems

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and insights

Storage Lens

Batch Operations

blog-output/

Copy S3 URI

Objects

Properties

Objects (5)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 > ⚙

☐	Name	Type	Last modified	Size	Storage class
☐	2026-06-18 22:29:01.txt	txt	June 18, 2026, 15:29:03 (UTC-07:00)	2.8 KB	Standard
☐	2026-06-18 22:34:29.txt	txt	June 18, 2026, 15:34:31 (UTC-07:00)	2.8 KB	Standard
☐	2026-06-18 22:37:05.txt	txt	June 18, 2026, 15:37:07 (UTC-07:00)	1.4 KB	Standard
☐	2026-06-18 22:43:14.txt	txt	June 18, 2026, 15:43:15 (UTC-07:00)	1.8 KB	Standard
☐	2026-06-18 22:45:18.txt	txt	June 18, 2026, 15:45:20 (UTC-07:00)	1.6 KB	Standard